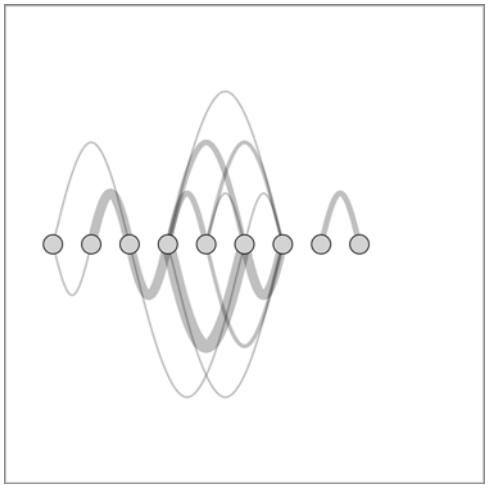


Figure 6.5 An arc diagram, with connections between nodes represented as arcs above and below the nodes. Arcs above the nodes indicate the connection is from left to right, while arcs below the nodes indicate the source is on the right and the target is on the left.

Notice that the edges array that we build uses a hash with the ID value of our edges to create object references. By building objects that have references to the source and target nodes, we can easily calculate the graphical attributes of the `<line>` or `<path>` element we're using to represent the connection. This is the same method used in the force layout that we'll look at later in the chapter. The result of the code is your first arc diagram, shown in figure 6.5.

With abstract charts like these, you're getting to the point where interactivity is no longer optional. Even though the links follow rules, and you're not dealing with too many nodes or edges, it can be hard to make out what is connected to what and how. You can add useful interactivity by having the edges highlight the connecting nodes on mouseover. You can also have the nodes highlight connected edges on mouseover by adding two new functions as shown in the following listing, with the results in figure 6.6.

Listing 6.6 Arc diagram interactivity

```
d3.selectAll("circle").on("mouseover", nodeOver);
d3.selectAll("path").on("mouseover", edgeOver);

function nodeOver(d,i) {
   d3.selectAll("circle").classed("active", function (p) {
               return p == d ? true : false;
   });

   d3.selectAll("path").classed("active", function (p) {
        return p.source == d || p.target == d ? true : false;
   });
};

function edgeOver(d) {
   d3.selectAll("path").classed("active", function(p) {
        return p == d ? true : false;
   });
```

Makes a selection of all nodes to set the class of the node being hovered over to "active"

Any edge where the selected node shows up as source or target renders as red